Program 10: Demonstrate Kubernetes Dashboard and CLI for Cluster Monitoring

Aim

To monitor and manage a Kubernetes cluster using both the **Kubernetes Dashboard (Web UI)** and the **Command Line Interface (kubectl)**.

Description

This experiment demonstrates how to monitor and manage a Kubernetes cluster using the Kubernetes Dashboard and CLI tools. The Dashboard provides a graphical interface to visualize and interact with cluster resources, while the CLI offers powerful commands to inspect, debug, and manage resources efficiently.

Project Structure

Program-10/
├── deployment.yaml
└── service.yaml

Prerequisites

- Minikube installed
- kubectl configured
- Basic understanding of Kubernetes resources

Step 1: Start Minikube with Specific Resources

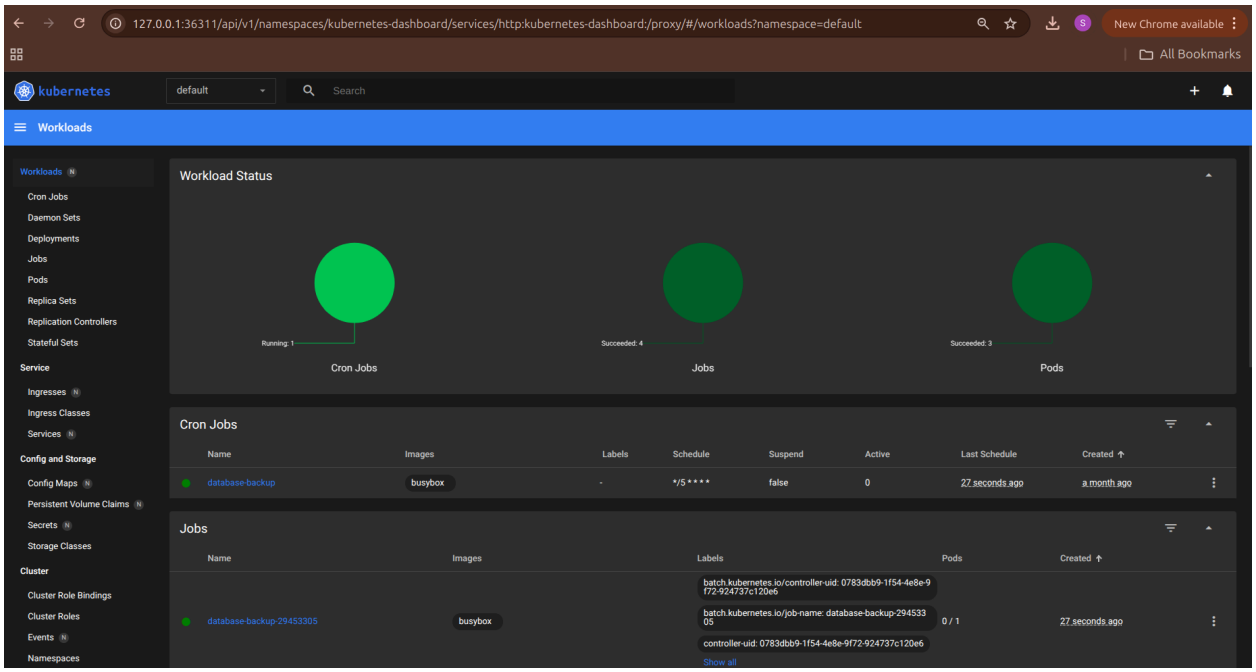
minikube start --cpus=2 --memory=4096

Explanation

- Starts a local Kubernetes cluster
- Allocates 2 CPUs and 4 GB RAM

Step 2: Enable Kubernetes Dashboard

minikube dashboard



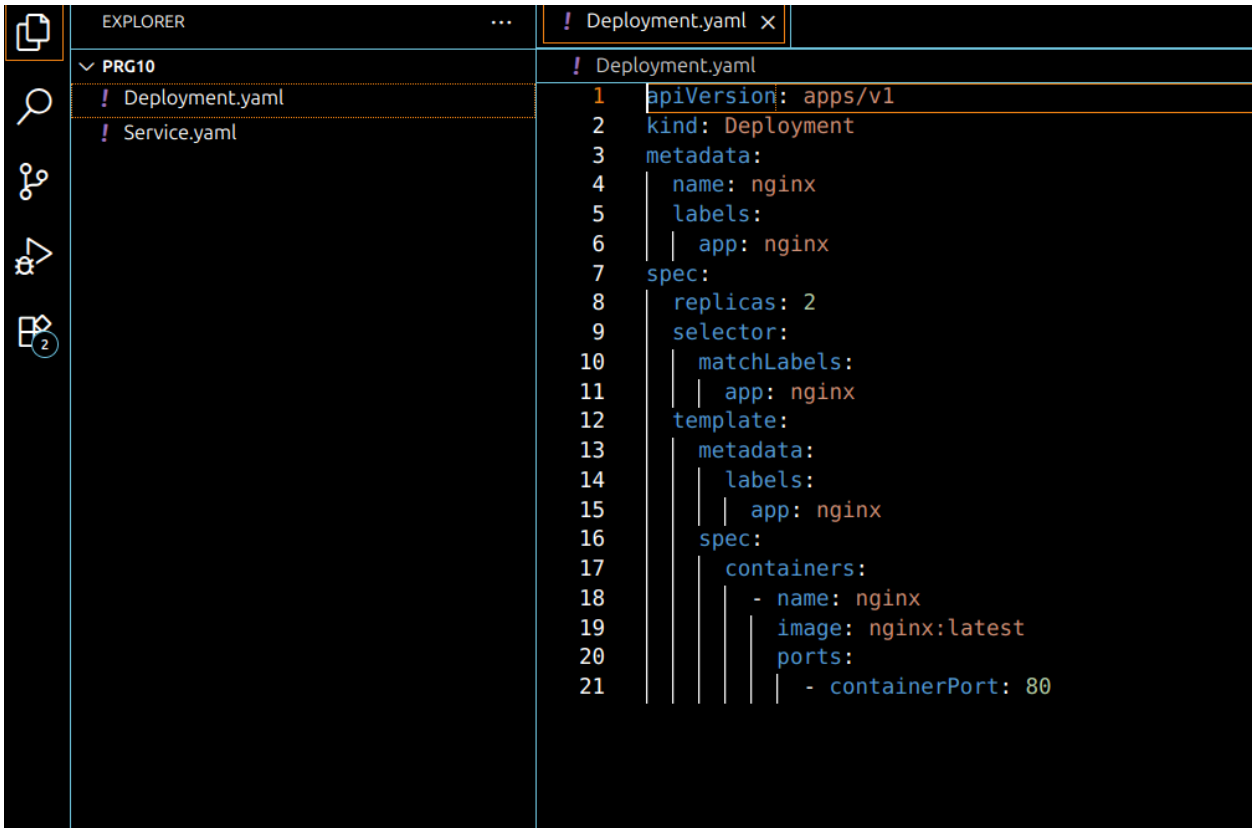
Dashboard Features

- View nodes, pods, deployments, and services
- Inspect pod logs and events
- Deploy applications from UI

Step 3: Create Deployment YAML (deployment.yaml)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
```

```
spec:
  replicas: 2
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:latest
        ports:
        - containerPort: 80
```

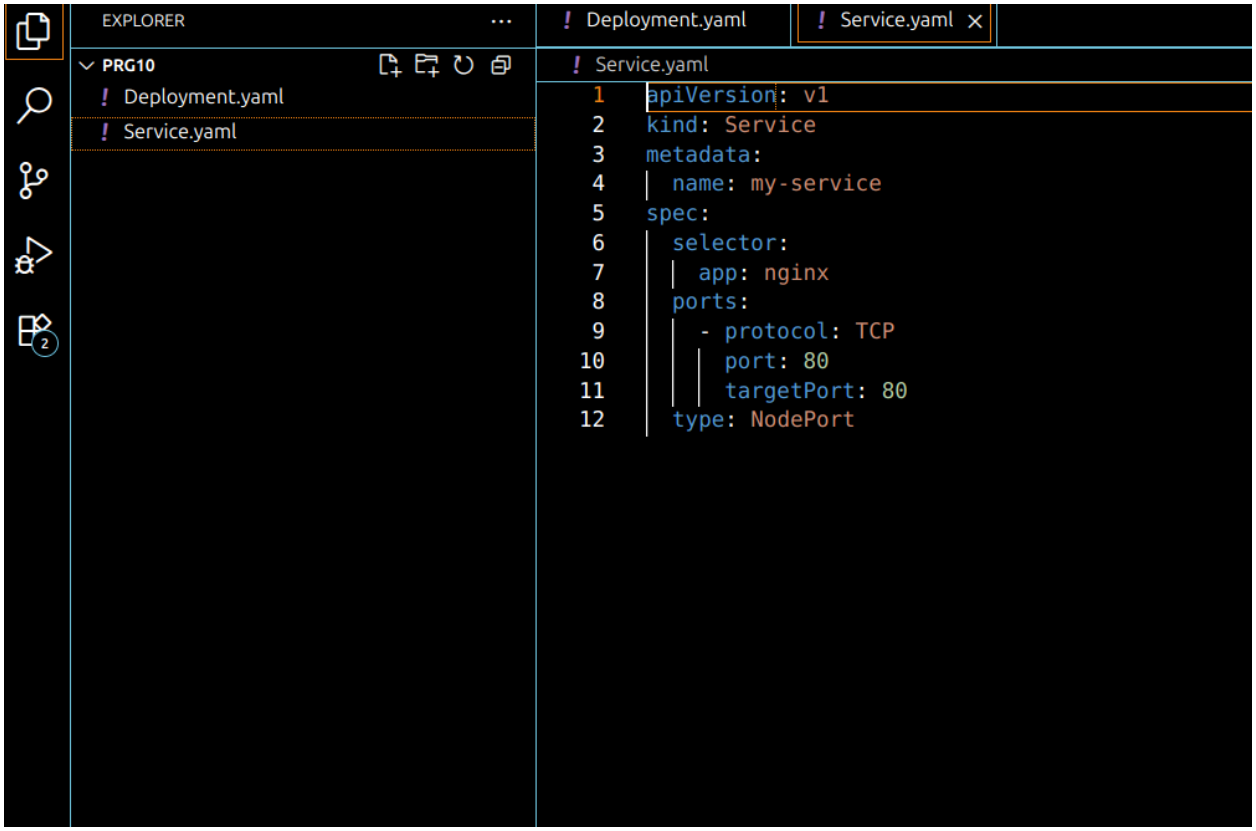


Explanation

- Deployment manages multiple Pods
- replicas ensures high availability
- template defines Pod configuration

Step 4: Create Service YAML (service.yaml)

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
spec:
  selector:
    app: nginx
  ports:
  - protocol: TCP
    port: 80
    targetPort: 80
  type: NodePort
```



Explanation

- Service exposes the application
- NodePort allows external access
- selector connects Service to Pods

Step 5: Apply Kubernetes Resources

kubectl apply -f deployment.yaml
kubectl apply -f service.yaml

Step 6: Monitor via CLI

kubectl get pods
kubectl get nodes
kubectl get svc

Step 7: Check Pod Logs

kubectl logs <pod-name>

Step 8: Describe Pod

kubectl describe pod <pod-name>

Step 9: Access the Application

minikube service nginx-service

The NGINX welcome page will open in the browser.

Step 10: Monitor Using Dashboard

- View deployment status
- Monitor pod health
- Inspect logs and events

Step 11: Clean-up

kubectl delete pods --all
kubectl delete svc --all
kubectl delete deployments --all

Expected Output

- NGINX Pods running successfully
- Service exposed via NodePort
- Application accessible in browser
- Cluster resources visible in Dashboard

Conclusion

This experiment demonstrates effective cluster monitoring using Kubernetes Dashboard and CLI commands by deploying and exposing an NGINX application.